DevMentor AI - By Dhruv Shukla

09 Aug 2026

Daily Learning Report

TODAY'S LEARNING

Control Statements

Day 3 | Beginner | 10 min read

Generated by DevMentor AI by Dhruv Shukla

? Today's Goal

By the end of today, Dhruv will be able to structure readable, deterministic, and high-performance control flow logic using modern C# control statements (if/else, modern switch expressions, guard clauses, and loops). You will learn how the Intermediate Language (IL) executes branching and how to eliminate deep nesting in enterprise ASP.NET Core applications.

? Core Concept

Control statements dictate the execution path of a program based on runtime evaluation. Without control statements, CPU execution moves purely sequentially from top to bottom.

The Problem It Solves

Applications must respond dynamically to varying state—such as user inputs, database responses, and HTTP status codes. Control flow structures enable conditional execution (selection) and repeated execution (iteration) safely and predictably.

How It Works Internally

At the machine level, the C# compiler translates control statements into IL jump instructions (br, brtrue, brfalse). For large conditional trees:

- if-else cascades: Evaluated linearly ($O(N)$ worst-case execution time).
- switch statements/expressions: Compiler evaluates key data types. For contiguous values or discrete strings, it often generates a binary search tree or direct branch table (jump table), resulting in $O(1)$ constant time lookup.

```
Linear standard branching (if-else):   [Cond 1] -> [Cond 2] -> [Cond 3] -> Target
Optimized jump table (switch):         [Value]  -> Jump Table Direct -> Target
```

Key Rules & Edge Cases

- Short-circuiting: In if (a && b), if a is false, b is never evaluated.
- Exhaustiveness: C# switch expressions MUST cover all possible inputs (or include a wildcard discard _), or the runtime throws SwitchExpressionException.
- Iteration Modification: You cannot modify an IEnumerable collection during foreach iteration because the underlying IEnumerator loses reference sync.

What Happens If Done Wrong

- Arrow Anti-Pattern: Deeply nested if blocks lead to high cyclomatic complexity, unreadable code, and high bug density.
- CPU Exhaustion: Improperly bounded while or for loops lock worker threads and starve the thread pool.

Complete Runnable C# Example

```
using System;

namespace ControlStatementsDemo
{
    public class Program
    {
        public static void Main()
        {
            int requestCount = 42;
            string userRole = "Admin";

            // Modern pattern matching switch expression
            string accessLevel = (userRole, requestCount) switch
            {
                ("Admin", _) => "Full Access Granted",
                ("User", < 50) => "Standard Access Granted",
                ("User", >= 50) => "Rate Limit Exceeded",
                _ => "Access Denied"
            };

            Console.WriteLine($"Access Level Result: {accessLevel}");

            // Loop control with break/continue
            for (int i = 1; i <= 5; i++)
            {
                if (i == 2) continue; // Skip iteration
                if (i == 4) break;    // Terminate loop early
```

```
            Console.WriteLine($"Processing item: {i}");
        }
    }
  }
}
```

🏢 Real Project Example

In an ASP.NET Core payment pipeline, processing an order requires validating parameters, checking user account tier, and routing through guard clauses before hitting business logic.

```
using System;

namespace RetailApi.Services
{
    public enum OrderStatus { Pending, Approved, Flagged, Processing }

    public class PaymentProcessor
    {
        public string EvaluateOrderRisk(decimal amount, bool isFirstTimeUser, OrderStatus status)
        {
            // Guard clause 1: Validation
            if (amount <= 0)
            {
                throw new ArgumentOutOfRangeException(nameof(amount), "Amount must be positive.");
            }

            // Guard clause 2: Terminal State Check
            if (status == OrderStatus.Flagged)
            {
                return "REJECT: Order is manually flagged for fraud review.";
            }

            // Switch expression with tuple matching & relational patterns
            return (amount, isFirstTimeUser, status) switch
            {
                ( > 10000m, true, _) => "HIGH_RISK: First time buyer large transaction",
                ( > 5000m, _, OrderStatus.Pending) => "MEDIUM_RISK: Requires secondary check",
                (_, _, OrderStatus.Approved) => "LOW_RISK: Order cleared",
                _ => "LOW_RISK: Standard processing flow"
            };
        }
    }
}
```

Explanation & Senior Architecture Insight

Guard Clauses: Check failure modes up front (amount <= 0 and status == OrderStatus.Flagged) and exit early using guard patterns. This reduces indentation levels to zero.

Switch Pattern Matching: Evaluates state contextually using tuples. The syntax is concise, expression-bodied, and fully type-safe.

Senior Design Standard: Avoid deep else blocks after guard clauses. Keep happy path execution flat against the left margin of the code editor.

⁉ Top 3 Mistakes

1. The Arrow Anti-Pattern (Deep Nesting)

Bad Code:

```
if (user != null) {
    if (user.IsActive) {
        if (user.HasSubscription) {
            ProcessPayment();
        }
    }
}
```

Why it fails: Creates massive cyclomatic complexity, making reading, debugging, and unit testing exceptionally difficult.

Good Fix (Guard Clauses):

```
if (user == null || !user.IsActive || !user.HasSubscription) return;

ProcessPayment();
```

---

2. Non-Exhaustive Switch Expressions

Bad Code:

```
string statusText = orderStatus switch
{
    OrderStatus.Pending => "Pending Verification",
```

```
    OrderStatus.Approved => "Payment Cleared"
}; // Missing default case or handling for Flagged/Processing
```

Why it fails: If orderStatus is OrderStatus.Flagged, C# throws a runtime System.Runtime.CompilerServices.SwitchExpressionException.

Good Fix:

```
string statusText = orderStatus switch
{
    OrderStatus.Pending => "Pending Verification",
    OrderStatus.Approved => "Payment Cleared",
    _ => "Status Unknown" // Discard wildcard handles remaining states
};
```

---

3. Modifying Collection During foreach Iteration

Bad Code:

```
foreach (var item in itemList)
{
    if (item.IsExpired)
    {
        itemList.Remove(item); // Throws exception!
    }
}
```

Why it fails: foreach uses IEnumerator internally. Modifying the underlying list mutates the version counter of the collection, invalidating the iterator.

Good Fix:

```
itemList.RemoveAll(item => item.IsExpired);
```

? Industry News

- Presentation: Keeping ChatGPT Fast as AI Development Accelerates

OpenAI engineering details performance strategies for maintaining real-time inference speeds while scaling complexity. This demonstrates how optimized control execution and memory layout directly dictate throughput

in high-load AI production systems.

Read full article

- Cloudflare's Precursor Detects Bots and AI Agents Through Continuous Behavioral Analysis

Cloudflare released Precursor, analyzing dynamic web request traffic via heuristic branching models. It emphasizes how low-latency control logic handles high-throughput packet evaluation at the edge.

Read full article

- GitHub Hardens npm and Actions Defaults, Drawing Debate over Delays versus Signing

GitHub changed default action behaviors to harden supply-chain security against script injections. Developers must structure deterministic verification paths in build pipelines to avoid workflow interruptions.

Read full article

- Cloudflare Launches Persistent, Stateful, Computer-like Environments for Agents

Cloudflare introduced stateful environments designed to maintain complex execution state machines across long-running background worker threads. Developers write predictable loop/control logic capable of serializing state reliably across distributed boundary layers.

Read full article

- Instacart Builds Blueberry, an AI-Powered Assistant to Help On-Call Engineers Investigate Incidents

Instacart built Blueberry to automate incident diagnosis using automated heuristic decision trees. Systems rely on clear, fault-tolerant execution flows to assist engineers rapidly during system failures.

Read full article

- Presentation: Rewriting All of Spotify's Code Base, All the Time

Spotify explains automated code migrations utilizing Abstract Syntax Trees (ASTs) to transform legacy conditional statements into modern constructs automatically, illustrating the enterprise value of standardized pattern matching.

Read full article

Interview Questions & Answers

Q1: What is the main difference between an if-else statement and a switch statement in C#?

A1: An if-else statement evaluates dynamic Boolean expressions sequentially, giving $O(N)$ branch complexity. A switch evaluates discrete values against constant patterns. The C# compiler can compile switch statements into direct jump tables ($O(1)$ efficiency) or binary search trees ($O(\log N)$) in Intermediate Language (IL).

Q2: What is the difference between break and continue inside loop statements?

A2: The break statement immediately terminates execution of the nearest enclosing loop or switch block, jumping execution outside the loop block. The continue statement skips the remaining code within the current iteration and jumps directly to the loop's next evaluation cycle.

```
for (int i = 0; i < 5; i++) {
    if (i == 1) continue; // Skips printing 1
    if (i == 3) break;    // Stops loop entirely at 3
}
```

Q3: How does a C# 8+ switch expression differ from a traditional switch statement?

A3: A traditional switch statement contains multiple case labels, requires explicit break; jump statements, and operates as a statement (executing side effects). A switch expression uses expression-bodied syntax (=>), returns a single value directly, relies on pattern matching, and enforces exhaustive handling through mandatory default handling (_).

Q4: Why does modifying a collection inside a foreach loop raise an InvalidOperationException?

A4: foreach loops rely on C#'s IEnumerator interface. Enumerators maintain an internal state version marker tied to the collection. When .Add() or .Remove() is called on the collection during iteration, the internal version counter increments. The next call to IEnumerator.MoveNext() detects a version mismatch and throws an InvalidOperationException to prevent unpredictable mutation behavior.

Q5: What are Guard Clauses, and how do they reduce Cyclomatic Complexity?

A5: Guard clauses are early validation checks placed at the beginning of a function that throw an exception or return early if conditions are not met. They eliminate nested if-else blocks (Arrow Anti-Pattern), keeping happy path logic unindented and flat. This directly minimizes cyclomatic complexity—the measure of linearly independent paths through code.

```
public void ProcessOrder(Order order) {
    if (order == null) throw new ArgumentNullException(nameof(order));
    if (!order.IsValid) return;

    // Core logic executed cleanly without nested blocks
}
```

Q6: How does pattern matching in C# switch expressions get compiled into Intermediate Language (IL)?

A6: The C# compiler translates complex switch expressions containing property, type, and relational patterns into optimized evaluation logic. Simple type checks compile using the IL instruction isinst. Relational checks (> 5) compile into conditional jump sequences (ble, bge). When checking string or integer constants, the compiler builds a dictionary or jump table hash lookup internally to avoid executing linear evaluation checks.

? Revision Summary

Topic: Day 2 — Operators in C#

- Arithmetic & Relational Operators: Fundamental building blocks (+, -, *, /, %, >, <). Watch out for integer division truncation (5 / 2 = 2)—cast operands to double or decimal when precision is required.
- Logical & Short-Circuit Operators: Conditional AND (&&) and OR (||) evaluate operands strictly left-to-right. Execution stops as soon as the outcome is finalized (false && expr immediately returns false without evaluating expr).
- Null-Coalescing Operators: Modern null handling relies on ?? (fallback value assignment) and ??= (assignment only if target is null).

One Thing To Remember: Operators form the expressions that control statements evaluate; short-circuiting operator rules directly safeguard your control flow from null reference exceptions (e.g., if (obj != null && obj.IsValid)).

? Tomorrow Preview

Tomorrow, we cover Methods and Functions in C#. You will learn how to encapsulate control flow into clean, modular, and reusable units of logic using parameters, return types, local functions, expression-bodied members, and extension methods.